Contraintes OCL

TIC315 — Projet 4, jeu d'échecs en ligne. Jalon J4.

Les contraintes portent sur le diagramme de classes du domaine (`diagrams/classes-domaine.puml`) et expriment les règles de gestion du cahier des charges que les multiplicités ne savent pas dire. Chacune donne la règle en français, son expression OCL et ce qu'elle ajoute au diagramme.

1. Conventions et hypothèses

- **Navigation.** On suit les noms de rôle du diagramme : `participants`, `parties`, `createur`, `invite`, `coups`, `depart`, `arrivee`, `pieceJouee`, `piecePrise`, `piecePromue`, `cases`, `pieces`. Une extrémité sans nom de rôle se désigne par le nom de sa classe en minuscules : `plateau`, `partie`, `cadence`, `classement`.
- **Classe-association.** Depuis une partie ou un joueur, `participation` donne les objets Participation. Depuis une Participation, chaque extrémité s'atteint par son nom de rôle : `p.participants` est le joueur, `p.parties` la partie.
- **Association qualifiée.** `j.classement[c]` est le classement du joueur `j` dans la catégorie `c`.
- **Types.** OCL n'a pas de type Date ni Duree : comme le cours, on suppose ces deux types munis des comparaisons usuelles. Les attributs `int` et `boolean` du diagramme sont les types OCL `Integer` et `Boolean`.
- **Préconditions et refus.** Une opération appelée depuis l'interface reçoit des demandes du réseau : elle ne fait pas confiance à l'appelant et vérifie elle-même ses préconditions (`verifierTrait`, `peutEtreRejointePar`, `peutJouer`). Une précondition non remplie produit le refus montré dans les diagrammes de séquence ; la postcondition décrit le cas où l'opération aboutit.

2. Opération auxiliaire

Trois contraintes ont besoin de savoir si une partie est « active », c'est-à-dire en attente ou en cours. On le définit une fois.

```
context Partie
def: estActive() : Boolean =
  self.etat = EtatPartie::EN_ATTENTE or self.etat = EtatPartie::EN_COURS
```

3. Joueurs

C1 — Pseudo et e-mail uniques (RG12)

Deux joueurs ne partagent ni un pseudo ni une adresse e-mail.

```
context Joueur
inv pseudoUnique: Joueur.allInstances()->isUnique(j | j.pseudo)
inv emailUnique:  Joueur.allInstances()->isUnique(j | j.email)
```

L'unicité porte sur l'ensemble des joueurs : aucune multiplicité ne peut l'exprimer, et `{unique}` ne concerne que les valeurs d'un attribut multivalué.

C2 — Une partie à la fois (RG11)

Un joueur participe à au plus une partie en attente ou en cours.

```
context Joueur
inv unePartieALaFois: self.parties->select(p | p.estActive())->size() <= 1
```

La multiplicité `0..*` côté `parties` est nécessaire pour l'historique (RG7) : seule une contrainte peut limiter les parties actives sans limiter les parties terminées.

4. Plateau

C3 — Les 64 cases de l'échiquier (aucune RG : règle du jeu)

Chaque case a une colonne de a à h et une rangée de 1 à 8, et deux cases d'un même plateau n'ont pas les mêmes coordonnées.

```
context Case
inv coordonneesValides:
  Set{'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h'}->includes(self.colonne)
  and self.rangee >= 1 and self.rangee <= 8

context Plateau
inv casesDistinctes:
  self.cases->forAll(c1, c2 |
    c1 <> c2 implies (c1.colonne <> c2.colonne or c1.rangee <> c2.rangee))
```

La multiplicité donne 64 cases ; ces deux invariants garantissent que ce sont les 64 cases de l'échiquier, chacune une seule fois.

5. Parties

C4 — Partie contre l'ordinateur : un seul joueur, sans pendule (RG10)

```
context PartieContreOrdinateur
inv unSeulParticipant: self.participants->size() = 1
inv sansPendule:       self.participation->forAll(p | p.tempsRestant.oclIsUndefined())
inv niTempsNiAccord:
  not self.motifFin.oclIsUndefined() implies
    self.motifFin <> MotifFin::TEMPS_ECOULE and self.motifFin <> MotifFin::ACCORD
```

L'association `1..2 participants` est définie sur Partie, pour les deux sortes de parties : la sous-classe la restreint à 1. Sans pendule, la partie ne peut pas se perdre au temps ; sans adversaire humain, personne ne propose la nulle.

C5 — Partie entre joueurs : créateur, invité et deux camps (RG3, RG13)

Le créateur participe toujours à sa partie ; il ne peut pas s'inviter lui-même ; tant que la partie attend, il est seul ; une fois commencée, elle oppose deux joueurs de couleurs différentes, dont l'invité s'il y en a un.

```
context PartieEntreJoueurs
inv createurParticipe: self.participants->includes(self.createur)
inv pasSaPropreInvitation:
  not self.invite.oclIsUndefined() implies self.invite <> self.createur
inv seulPendantLAttente:
  self.etat = EtatPartie::EN_ATTENTE implies self.participants = Set{self.createur}
inv deuxCampsUneFoisCommencee:
  (self.etat = EtatPartie::EN_COURS or self.etat = EtatPartie::TERMINEE) implies
    self.participants->size() = 2
    and self.participation->forAll(p | not p.couleur.oclIsUndefined())
    and self.participation->isUnique(p | p.couleur)
inv adversaireInvite:
  (not self.invite.oclIsUndefined() and self.participants->size() = 2)
    implies self.participants->includes(self.invite)
```

Le diagramme d'objets « impossible » (cas C) montre une partie que les multiplicités acceptent, avec deux participants blancs : `deuxCampsUneFoisCommencee` la refuse.

C6 — Résultat et motif, seulement pour une partie terminée (RG4, RG15)

Une partie a un résultat et un motif de fin si et seulement si elle est terminée ; une partie annulée n'en a donc pas. Le résultat doit être cohérent avec le motif.

```
context Partie
inv resultatSiTerminee:
  (self.etat = EtatPartie::TERMINEE) = not self.resultat.oclIsUndefined()
inv motifSiTerminee:
  (self.etat = EtatPartie::TERMINEE) = not self.motifFin.oclIsUndefined()
inv resultatCoherentAvecMotif:
  self.etat = EtatPartie::TERMINEE implies
    let motifsDeNulle : Set(MotifFin) =
      Set{MotifFin::PAT, MotifFin::ACCORD, MotifFin::TRIPLE_REPETITION,
          MotifFin::CINQUANTE_COUPS, MotifFin::MATERIEL_INSUFFISANT}
    in (motifsDeNulle->includes(self.motifFin) implies self.resultat = Resultat::NULLE)
       and ((self.motifFin = MotifFin::MAT or self.motifFin = MotifFin::ABANDON)
            implies self.resultat <> Resultat::NULLE)
```

Les multiplicités `[0..1]` disent seulement que les valeurs sont facultatives, pas quand elles doivent être présentes. TEMPS_ECOULE n'est pas contraint : il donne une défaite, ou une nulle si l'adversaire ne peut pas mater (RG5).

C7 — Les coups d'une partie (RG7)

Une partie en attente n'a aucun coup ; les coups sont numérotés 1, 2, 3… dans l'ordre où ils ont été joués.

```
context Partie
inv pasDeCoupAvantLeDebut:
  self.etat = EtatPartie::EN_ATTENTE implies self.coups->isEmpty()
inv coupsNumerotesDansLOrdre:
  self.coups->forAll(c | c.numero = self.coups->indexOf(c))
```

`{ordered}` dit que les coups sont rangés ; l'invariant relie ce rang à l'attribut `numero`, utilisé pour revoir une partie coup par coup (BF11).

6. Coups

C8 — Un coup respecte les camps et le plateau (RG1, RG2, RG3)

Les blancs jouent les coups impairs et les noirs les coups pairs ; une pièce part d'une case et arrive sur une autre, du plateau de sa partie ; on ne prend que des pièces adverses et jamais le roi ; seul un pion arrivé sur la dernière rangée est promu, en une pièce de sa couleur qui n'est ni un roi ni un pion.

```
context Coup
inv alternanceDesCamps:
  self.pieceJouee.couleur =
    (if self.numero.mod(2) = 1 then Couleur::BLANC else Couleur::NOIR endif)
inv casesDuPlateauDeLaPartie:
  self.depart <> self.arrivee
  and self.partie.plateau.cases->includes(self.depart)
  and self.partie.plateau.cases->includes(self.arrivee)
  and self.partie.plateau.pieces->includes(self.pieceJouee)
inv priseDUnePieceAdverse:
  not self.piecePrise.oclIsUndefined() implies
    self.piecePrise.couleur <> self.pieceJouee.couleur
    and not self.piecePrise.oclIsKindOf(Roi)
inv promotionDUnPion:
  not self.piecePromue.oclIsUndefined() implies
    self.pieceJouee.oclIsTypeOf(Pion)
    and self.arrivee.rangee =
      (if self.pieceJouee.couleur = Couleur::BLANC then 8 else 1 endif)
    and self.piecePromue.couleur = self.pieceJouee.couleur
    and not self.piecePromue.oclIsKindOf(Roi)
    and not self.piecePromue.oclIsKindOf(Pion)
```

Les associations disent qu'un coup a une case de départ, une case d'arrivée et des pièces ; elles ne disent pas lesquelles sont permises. La légalité complète d'un déplacement (trajectoire, échec au roi) reste calculée par `Plateau::estLegal`.

7. Attributs dérivés et requêtes

C9 — Qui a le trait (RG3)

Les blancs commencent : le trait est aux blancs après un nombre pair de coups.

```
context Partie::trait : Couleur
derive: if self.coups->size().mod(2) = 0 then Couleur::BLANC else Couleur::NOIR endif
```

C10 — Une pièce a déjà bougé (RG2)

Une pièce a déjà bougé si elle a été jouée dans un coup de sa partie. Le roque et l'avance de deux cases du pion en dépendent.

```
context Piece::aDejaBouge : Boolean
derive: self.plateau.partie.coups->exists(c | c.pieceJouee = self)
```

C11 — Un joueur peut-il jouer ? (RG8, RG11)

```
context Joueur::peutJouer() : Boolean
body: self.etat = EtatCompte::ACTIF and not self.parties->exists(p | p.estActive())
```

Les séquences « rejoindre une partie » et « contre l'ordinateur » appellent `peutJouer()` : cette définition fixe exactement ce qu'elle vérifie.

8. Contrats d'opérations

C12 — Jouer un coup (RG1, RG3, RG7)

```
context Partie::jouerCoup(joueur : Joueur, depart : Case, arrivee : Case,
                          promotion : TypePromotion) : Coup
pre partieEnCours:   self.etat = EtatPartie::EN_COURS
pre joueurParticipe: self.participants->includes(joueur)
pre aLeTrait:
  self.participation->any(p | p.participants = joueur).couleur = self.trait
pre coupLegal:       self.plateau.estLegal(depart, arrivee)
post unCoupDePlus:   self.coups->size() = self.coups@pre->size() + 1
post coupEnregistre:
  result = self.coups->last()
  and result.depart = depart and result.arrivee = arrivee
post traitPasse:
  self.etat = EtatPartie::EN_COURS implies self.trait <> self.trait@pre
```

C13 — Rejoindre une partie (RG3, RG8, RG11, RG13)

```
context PartieEntreJoueurs::rejoindre(joueur : Joueur)
pre enAttente:          self.etat = EtatPartie::EN_ATTENTE
pre pasLeCreateur:      joueur <> self.createur
pre inviteSiInvitation: not self.invite.oclIsUndefined() implies joueur = self.invite
pre joueurDisponible:   joueur.peutJouer()
post partieDemarree:
  self.etat = EtatPartie::EN_COURS and not self.dateDebut.oclIsUndefined()
post deuxParticipants:  self.participants = Set{self.createur, joueur}
```

C14 — Terminer une partie : seul le bon classement change (RG6)

À la fin d'une partie entre joueurs, seul le classement de la catégorie de sa cadence peut changer ; à la fin d'une partie contre l'ordinateur, aucun ne change.

```
context Partie::terminer(resultat : Resultat, motif : MotifFin)
pre enCours: self.etat = EtatPartie::EN_COURS
post terminee:
```

```
  self.etat = EtatPartie::TERMINEE
  and self.resultat = resultat and self.motifFin = motif
post seulLeClassementDeLaCadenceChange:
  self.participants->forAll(j |
    Set{CategorieCadence::BULLET, CategorieCadence::BLITZ, CategorieCadence::RAPIDE}
      ->forAll(c |
        (self.oclIsTypeOf(PartieContreOrdinateur)
         or c <> self.oclAsType(PartieEntreJoueurs).cadence.categorie)
        implies j.classement[c].points = j.classement[c].points@pre))
```

La formule de calcul des points (patron Stratégie) n'est pas fixée ici : le contrat dit seulement quels classements elle a le droit de modifier.

C15 — Suspendre un compte (RG8, RG14)

Après la suspension, le joueur n'a plus de partie active, et son compte existe toujours.

```
context Joueur::suspendre()
pre actif:              self.etat = EtatCompte::ACTIF
post suspendu:          self.etat = EtatCompte::SUSPENDU
post plusDePartieActive: not self.parties->exists(p | p.estActive())
post compteConserve:    Joueur.allInstances()->includes(self)
```

`plusDePartieActive` résume RG8 : sa partie en attente est annulée, sa partie en cours est terminée par son abandon. `compteConserve` exprime RG14 : suspendre n'est pas supprimer.

9. Récapitulatif

N°	Sorte	Contexte	Règles
—	def	Partie::estActive()	RG11
C1	inv (2)	Joueur	RG12
C2	inv	Joueur	RG11
C3	inv (2)	Case, Plateau	Règle du jeu
C4	inv (3)	PartieContreOrdinateur	RG5, RG10
C5	inv (5)	PartieEntreJoueurs	RG3, RG13
C6	inv (3)	Partie	RG4, RG5, RG15
C7	inv (2)	Partie	RG7
C8	inv (4)	Coup	RG1, RG2, RG3
C9	derive	Partie::trait	RG3
C10	derive	Piece::aDejaBouge	RG2
C11	body	Joueur::peutJouer()	RG8, RG11
C12	pre (4), post (3)	Partie::jouerCoup()	RG1, RG3, RG7
C13	pre (4), post (2)	PartieEntreJoueurs::rejoindre()	RG3, RG8, RG11, RG13
C14	pre, post (2)	Partie::terminer()	RG6
C15	pre, post (3)	Joueur::suspendre()	RG8, RG14

Soit 15 contraintes pour 8 exigées. Elles utilisent six des sept mots-clés du cours : `def`, `inv`, `derive`, `body`, `pre` et `post` ; seul `init` est inutile, les valeurs initiales figurant déjà dans le diagramme (`etat = EN_ATTENTE`, `etat = ACTIF`).